

Cross-Lingual Music Recommendation Using Hybrid TF-IDF and Multilingual Sentence Embeddings

CS 5100 — Foundations of Artificial Intelligence

Final Project Report

Jieyao Chen

Shivam Singh

April 8, 2026

Contents

1	Introduction	3
2	Methods	5
2.1	Dataset Preparation	5
2.2	TF-IDF Recommender	6
2.3	Multilingual Embedding Recommender	6
2.4	Hybrid Recommender	7
2.5	Explainability	8
2.6	Evaluation Protocol	8
2.7	Visualization	9
2.8	System Interface	9
3	Results	10
3.1	Quantitative Evaluation	10
3.2	Sample Output and Qualitative Comparison	11
3.3	System Screenshots	12
3.4	Hybrid Mode Coverage	13
4	Conclusions	13
	Works Cited	16

1 Introduction

Music recommendation has become one of the most commercially and culturally significant applications of artificial intelligence. Platforms such as Spotify, Apple Music, and YouTube Music serve hundreds of millions of users daily, each relying on recommendation algorithms to surface songs that a listener is likely to enjoy. Despite this ubiquity, the problem of music recommendation remains genuinely difficult. Unlike product recommendation, where user preferences can be inferred from explicit purchase history, musical taste is shaped by subtle, often subjective factors: lyrical theme, emotional tone, rhythmic energy, cultural context, and language. A listener who enjoys a melancholic Portuguese fado may also enjoy a Japanese enka ballad, yet no surface-level keyword match would reveal this connection. Building systems that capture this semantic depth is one of the central challenges of the field.

This project addresses the music recommendation problem from three complementary angles. The first approach is a classical text-based method: a TF-IDF (Term Frequency–Inverse Document Frequency) recommender that represents each song’s lyrics as a sparse vector of weighted word frequencies and identifies similar songs by measuring cosine similarity between these vectors. This approach is interpretable and efficient but is limited to lexical overlap; it cannot generalize across languages or capture meaning beyond shared vocabulary.

The second approach uses multilingual sentence embeddings generated by a pre-trained transformer-based model, specifically the `paraphrase-multilingual-MiniLM-L12-v2` model from the Sentence Transformers library (Reimers & Gurevych, 2019). This model maps lyrics from any language into a shared 384-dimensional vector space, enabling cross-lingual similarity matching. Two songs with semantically similar themes—such as longing, celebration, or defiance—will be placed near each other in this space even if they are written in entirely different languages. Approximate nearest-neighbor search is performed using FAISS (Johnson et al., 2019), an efficient similarity search library developed by Meta AI, making retrieval

fast even over tens of thousands of songs.

The third approach is a hybrid recommender that combines TF-IDF lyric vectors with standardized audio features—including tempo, energy, and musical valence—drawn from a Spotify audio features dataset. By concatenating lyric and audio representations before computing similarity, the hybrid model can identify songs that are both thematically and acoustically similar, addressing a limitation common to purely text-based systems.

Prior work on music recommendation has approached the problem from many directions. Collaborative filtering methods (Koren et al., 2009) leverage user listening histories to find patterns across users, but are subject to the cold-start problem and require large-scale behavioral data. Content-based methods, which use item features rather than user behavior, are more suitable when such data is unavailable. Celma & Herrera (2008) demonstrated that combining acoustic content descriptors with network-based popularity signals improves recommendation diversity. More recently, work in natural language processing has shown that pre-trained language models can be adapted to represent long-form text—including song lyrics—in ways that capture latent semantic meaning (Devlin et al., 2019). This project draws on this tradition by using a multilingual variant of a sentence transformer to represent lyric content, extending the approach to a multilingual corpus.

The dataset used in this project is the Spotify Million Song Dataset, a publicly available collection of 57,650 songs with lyrics from 643 artists, sourced from Kaggle. The average lyric length is approximately 116 words per song after preprocessing, reflecting a corpus that is rich enough for meaningful text similarity computation. The dataset covers a wide variety of genres and languages, making it a strong testbed for cross-lingual recommendation. Audio feature data (energy, valence, tempo, danceability, and others) was obtained from a companion Spotify audio features dataset, also sourced from Kaggle, containing approximately 130,000 unique tracks. An inner join on artist name and song title yielded a matched subset used for hybrid model training.

The full system is deployed as an interactive Streamlit web application that allows users to select a seed song, choose a recommendation mode, and optionally enter a free-text semantic query to retrieve songs by mood or theme rather than by title. The application supports paginated browsing through thousands of recommendations, context-aware filters by mood and energy level, an explainability panel showing why each song was recommended, and both 3D and 2D embedding space visualizations. The application demonstrates that content-based recommendation with deep language models can produce musically coherent, cross-lingual recommendations without any user behavioral data.

2 Methods

2.1 Dataset Preparation

The primary dataset is the Spotify Million Song Dataset (`spotify_millsongdata.csv`), containing 57,650 songs from 643 artists with raw lyrics text. A standard preprocessing pipeline was applied: punctuation and numbers were removed using regular expressions, text was lowercased, and English stopwords were removed using the NLTK library (Bird et al., 2009). The resulting `cleaned_text` field, averaging approximately 116 words per song, was used as input to both the TF-IDF vectorizer and the embedding model. Processed data was saved as a Parquet file (`data/processed/cleaned_lyrics.parquet`) for fast reloading on subsequent runs.

Audio feature data was obtained from a Kaggle dataset containing Spotify chart tracks from 2018–2019 with associated audio features. Two CSV files—covering April 2019 and November 2018—were concatenated and deduplicated on Spotify track ID. The resulting table was standardized to use consistent column names (`artist`, `song`, plus numeric feature columns) using a custom normalization module (`app/audio_features.py`). Audio rows were joined to the lyrics table using a normalized join key (lowercased, whitespace-stripped artist

and song name). The matched subset contained approximately 325 songs out of 57,650, representing a match rate of roughly 0.6%. This low overlap is a consequence of naming convention differences between the two sources: the Spotify chart data frequently includes parenthetical suffixes such as “(Remastered)” or “(Radio Edit)”, abbreviates featured artist credits differently (e.g., “feat.” vs. “ft.” vs. “featuring”), and in some cases uses alternate transliterations of non-English artist names. Several normalization strategies were applied—including lowercasing, punctuation stripping, and whitespace normalization—but these were insufficient to resolve differences in song title structure without risking false positive matches. Resolving the full mismatch would require either a track ID-based join via the Spotify Web API or a fuzzy string matching approach, both of which were considered out of scope for this implementation.

2.2 TF-IDF Recommender

The TF-IDF recommender (`app/tfidf_recommender.py`) vectorizes the `cleaned_text` column of all songs using scikit-learn’s `TfidfVectorizer` (Pedregosa et al., 2011) with a vocabulary limited to the 5,000 most frequent terms. Given a seed song, the model computes the cosine similarity between the seed song’s TF-IDF vector and all other songs in the corpus, then returns the top- N most similar songs ranked by descending similarity score. This approach is fully interpretable: shared high-weight terms between the seed and recommended songs can be shown directly to the user as an explanation. The `explain_tfidf_overlap` function in `app/explainability.py` extracts these shared keywords and displays them as a reason beneath each recommendation card in the interface.

2.3 Multilingual Embedding Recommender

The embedding recommender (`app/embedding_recommender.py`) uses the multilingual-MiniLM-L12-v2 model, a distilled multilingual sentence transformer pre-trained on more than one billion sentence pairs across 50+ languages. Each song’s `cleaned_text` is encoded into a

384-dimensional dense vector with L2 normalization applied. Embeddings are computed in batches of 64 and stored as a NumPy array. An FAISS `IndexFlatIP` (inner product index) is built over the normalized embeddings, enabling efficient approximate nearest-neighbor search equivalent to cosine similarity. During recommendation, the seed song’s embedding is retrieved by title lookup, and the top- N nearest neighbors are returned from the FAISS index.

A key feature of this mode is free-text semantic search: rather than specifying a seed song, the user can type any mood, theme, or lyric fragment—for example, “heartbreak at midnight” or “upbeat summer road trip”—and the model encodes the query text into the same embedding space, returning the nearest songs by semantic meaning. This enables entirely query-driven discovery that requires no knowledge of specific song titles and is language-agnostic. Embeddings for all 57,650 songs are pre-computed and saved to disk alongside a metadata table, so subsequent runs load the artifacts rather than recomputing them.

2.4 Hybrid Recommender

The hybrid recommender (`app/hybrid_recommender.py`) combines TF-IDF lyric features with standardized audio features. It fits a TF-IDF vectorizer on the `cleaned_text` of matched songs, independently scales the audio feature columns (tempo, energy, valence) using scikit-learn’s `StandardScaler`, and concatenates the sparse TF-IDF matrix with the dense scaled audio matrix using `scipy.sparse.hstack`. Cosine similarity is computed over this joint representation. Because the hybrid dataset is limited to the approximately 325 songs for which both lyrics and matched audio features exist, the system falls back gracefully to TF-IDF on the full corpus when a requested seed song is not present in the hybrid subset. The interface notifies the user of this fallback with an inline message, preserving a smooth user experience without surfacing an error.

2.5 Explainability

To make recommendations interpretable, the system includes an explainability module (`app/explainability.py`). For TF-IDF-based recommendations, shared high-weight terms between the seed and recommended song are extracted and displayed beneath each recommendation card. For hybrid and embedding modes, a metadata-based explanation notes shared attributes—such as artist, detected language, energy level, or valence—between the seed song and each recommendation. This design ensures that users can understand the basis for every recommendation regardless of which model produced it.

2.6 Evaluation Protocol

Quantitative evaluation was performed using a leave-one-out protocol implemented in `scripts/evaluate_recommenders.py`. For each query song, the “relevant” set was defined as all other songs by the same artist in the dataset. The intuition is that songs by the same artist share thematic, stylistic, and lyrical similarities, making artist-level co-occurrence a reasonable proxy for ground-truth relevance in the absence of explicit user ratings. This proxy is consistent with the approach used in prior work on content-based music retrieval (Schedl et al., 2014).

Three metrics were computed at $k = 5$ over 200 randomly sampled query songs:

- **Precision@ k** : fraction of top- k recommendations that are relevant.
- **Recall@ k** : fraction of relevant items appearing in the top- k .
- **NDCG@ k** : normalized discounted cumulative gain, which rewards relevant items appearing at higher ranks.

Both the TF-IDF and embedding recommenders were evaluated; the hybrid recommender was excluded from this evaluation because its coverage of only approximately 325 songs is insufficient for a statistically meaningful comparison on 200 queries.

2.7 Visualization

A 3D visualization module (`app/streamlit_app.py`, `compute_umap_coords`) projects the full embedding space into three dimensions using Principal Component Analysis (PCA). An earlier iteration of the system used UMAP (McInnes et al., 2018) for dimensionality reduction, which produces more locally faithful cluster structure but caused instability in the Streamlit runtime due to Numba JIT compilation conflicts; PCA was adopted as the stable alternative. The 3D visualization is rendered as an interactive Plotly scatter plot in which the seed song is highlighted in red, recommended songs are shown in yellow, and the remaining corpus appears in blue. A separate 2D cluster map using the same PCA decomposition provides a complementary overhead view of the embedding space, making it easier to identify large-scale thematic regions. Both visualizations allow users to directly observe the geometric structure of the embedding space and to verify that the seed and recommended songs occupy nearby regions in the latent space.

2.8 System Interface

The complete system is deployed as an interactive Streamlit web application. A compact header bar displays dataset statistics—57,650 songs, 643 artists, approximately 116 average words per song—alongside a mode badge and a plain-language description of the currently active recommendation strategy. Users select a seed song and recommendation mode from the sidebar, and may optionally apply mood or minimum-energy filters. Recommendations are displayed as ranked cards in a paginated layout with Previous, Next, and Shuffle navigation controls that enable browsing across the full candidate set. Each card shows the recommendation rank, song title, artist, similarity score, and an explanation of why the song was recommended. A similarity score bar chart on the right provides a visual summary of the distribution of scores in the current page of results. The 3D and 2D embedding visualizations are available as toggleable panels below the main results.

3 Results

3.1 Quantitative Evaluation

The evaluation results for the TF-IDF and Embedding recommenders, computed over 200 query songs with $k = 5$, are summarized in Table 1.

Table 1: Quantitative evaluation results at $k = 5$ over 200 query songs.

Model	Precision@5	Recall@5	NDCG@5
TF-IDF	0.030	0.00153	0.0334
Embedding	0.020	0.00092	0.0225

The TF-IDF model achieved higher precision, recall, and NDCG than the embedding model under this evaluation protocol. This result may initially seem surprising, given that the embedding model is a more sophisticated approach. However, this outcome is consistent with a known limitation of the artist-based relevance proxy. When ground-truth relevance is defined as “same artist,” lexical similarity is an extremely strong signal, because many artists write lyrics with consistent vocabulary, themes, and style. TF-IDF vectors will naturally cluster songs by the same artist because those songs share stopword-filtered vocabulary. The embedding model, by contrast, captures cross-artist semantic similarity—finding songs that are thematically similar regardless of artist—which is arguably more useful for music discovery but is penalized under a same-artist relevance proxy.

The absolute values of all three metrics are low, which reflects the difficulty of the evaluation task: the relevant set for each query typically contains only a small number of songs (between 2 and 20, depending on the artist), and the top-5 recommendations must overlap with this set. In a dataset with 57,650 songs, random chance would yield a precision near 0.0001 for an artist with 5 other songs, so the observed precision of 0.030 represents a meaningful 30× improvement over chance.

3.2 Sample Output and Qualitative Comparison

To illustrate the qualitative differences between models, Table 2 shows the top-5 recommendations produced by each model for the seed song “Lose Yourself” by Eminem. This song presents a useful test case because its lyrics center on a specific narrative of seizing a critical opportunity under pressure—themes that differ substantially from Eminem’s other work in the dataset.

Table 2: Top-5 recommendations for seed song “Lose Yourself” (Eminem).

Rank	TF-IDF	Score	Embedding	Score
1	Fate Of Man (Glen Campbell)	0.330	Five Becomes Four (Yellowcard)	0.725
2	A Well Respected Man (Kinks)	0.325	On My Own (The Used)	0.716
3	Off He Goes (Pearl Jam)	0.320	Dancin’ (Vanilla Ice)	0.715
4	Straight For The Heart (Toto)	0.318	I Lived (OneRepublic)	0.713
5	Bamboo Bimbo (Devo)	0.309	Shoot From The Hip (W.A.S.P.)	0.711

The TF-IDF results illustrate a characteristic failure mode of lexical similarity: the recommended songs share individual cleaned tokens with “Lose Yourself” but do not reflect the song’s thematic content. Low cosine scores (0.30–0.33) indicate that the vocabulary overlap is sparse and largely coincidental. The embedding model, by contrast, produces recommendations with substantially higher similarity scores (0.71–0.73) and returns songs that share thematic elements—perseverance, self-determination, and emotional intensity—regardless of artist or shared vocabulary. OneRepublic’s “I Lived” and The Used’s “On My Own” are thematically coherent recommendations for a song about fighting through adversity, even though neither artist shares Eminem’s stylistic vocabulary.

Table 3 shows results for a second seed song, “Bohemian Rhapsody” by Queen, which presents a different challenge: its lyrics are highly distinctive and structurally unusual.

Table 3: Top-5 recommendations for seed song “Bohemian Rhapsody” (Queen).

Rank	TF-IDF	Score	Embedding	Score
1	Bohemian Polka (Weird Al Yankovic)	0.973	Bohemian Polka (Weird Al Yankovic)	0.996
2	Let Yourself Go (Green Day)	0.509	Ain’t Nothing Like The Real Thing (Elton John)	0.780
3	Who Let You Go (The Killers)	0.475	Never Let You Go (Justin Bieber)	0.774
4	Let Me Go (Avril Lavigne)	0.475	Be For Real (Leonard Cohen)	0.767
5	Let Me Go Easy (Indigo Girls)	0.466	No. 1 Hit Medley (Modern Talking)	0.764

Both models correctly identify “Bohemian Polka” by Weird Al Yankovic as the most similar song, since it is a direct lyric parody with near-identical text. Beyond this obvious match, TF-IDF recommendations cluster around songs sharing the word “go” or “let”—a surface-level lexical artifact—while the embedding model surfaces songs from artists with stylistically similar dramatic or theatrical qualities (Elton John, Leonard Cohen), reflecting genuine thematic similarity rather than token overlap.

The 3D PCA visualization confirms these patterns: the embedding space groups songs by thematic and stylistic similarity, and the seed song’s nearest neighbors in the visualization correspond closely to the top recommendations returned by the embedding model. In the 2D cluster map, distinct regions are visible that correspond loosely to genre and emotional register, suggesting that the multilingual model has captured meaningful structure in the lyric space even without any explicit genre labels.

3.3 System Screenshots

Figure 1 shows the main recommendation interface of the deployed Streamlit application, demonstrating the TF-IDF mode with shared lyric keyword explanations displayed beneath each recommendation card.

3.4 Hybrid Mode Coverage

As noted in the Methods section, the hybrid mode is limited by the 0.6% overlap between the lyric dataset and the audio features dataset. When a user selects Hybrid mode for a song not in this matched subset, the system falls back to TF-IDF on the full 57,650-song corpus and displays an inline notification. This design ensures that the application remains fully functional for all songs while clearly communicating when full hybrid scoring is not available. A dataset providing audio features specifically matched to the Million Song lyric corpus—for example, via Spotify Web API lookup using track IDs—would substantially increase coverage and allow the hybrid recommender to be evaluated quantitatively alongside the other two models.

4 Conclusions

This project demonstrates that content-based music recommendation can be implemented effectively using a combination of classical NLP techniques and modern pre-trained language models. The three-mode architecture—TF-IDF, multilingual embeddings, and hybrid—illustrates a progression of increasing sophistication, each with distinct strengths and trade-offs.

The TF-IDF recommender is computationally inexpensive, fully interpretable, and achieves strong same-artist recall. It is well-suited for applications where the primary goal is to surface songs by the same artist or in the same stylistic tradition. Its key weakness is that it cannot generalize beyond shared vocabulary: two songs with identical themes but different word choices—for example, one in English and one in Spanish—will receive a similarity score of zero.

The multilingual embedding recommender addresses this limitation directly. By representing songs in a shared semantic space regardless of language, it enables genuinely cross-lingual

music discovery. The semantic search feature, which allows users to query by mood or theme rather than by song title, further extends this capability: a user can type “late night melancholy” and receive semantically relevant recommendations without knowing any specific song names. The embedding model’s lower performance under the artist-based evaluation protocol should not be interpreted as evidence of lower quality; it reflects the fact that the embedding model is solving a different and arguably more valuable version of the recommendation problem.

The hybrid recommender represents the most complete approach to the problem, incorporating both semantic content and acoustic features. It is limited in the current implementation by data availability, but the architecture is correct: the concatenation of standardized audio and TF-IDF features into a joint similarity matrix is a principled way to combine modalities, and the graceful fallback to TF-IDF when audio features are unavailable ensures that the system remains functional across the full catalog. The inline fallback notification in the interface—informing users when hybrid scoring is not available for a given song—exemplifies the transparency-first design principle applied throughout the application.

Several directions for future improvement are apparent. First, audio feature coverage could be expanded by querying the Spotify Web API directly using track IDs, which would allow audio features to be retrieved for all or most of the 57,650 songs in the lyric corpus and enable quantitative evaluation of the hybrid model. Second, the evaluation protocol could be improved by collecting explicit relevance judgments from human evaluators or by using listening history data as a behavioral signal, both of which would provide a more accurate measure of recommendation quality than the artist-based proxy. Third, the embedding model could be fine-tuned on a music-specific corpus using contrastive learning (Chen et al., 2020), potentially improving its ability to distinguish musical similarity from general semantic similarity. Fourth, the system could be extended to incorporate collaborative filtering signals if user behavior data becomes available, combining the strengths of content-based and

collaborative approaches in the manner described by Burke (2002). Finally, replacing the current PCA-based visualization with UMAP (McInnes et al., 2018), once runtime stability issues are resolved, would yield a more locally faithful projection of the embedding space; preliminary experimentation with UMAP revealed tighter and more interpretable thematic clusters than those produced by PCA, suggesting that the 2D and 3D visualizations could more clearly communicate genre and mood structure to end users.

In summary, this project successfully implements and evaluates three content-based music recommendation approaches, deploys them in a fully functional interactive Streamlit application featuring cross-lingual semantic search, explainable recommendations, paginated navigation, and dual 2D/3D embedding visualizations, and quantitatively compares two of the three approaches using standard information retrieval metrics. The results highlight the complementary strengths of lexical and semantic similarity, and the system as a whole demonstrates how pre-trained multilingual language models can be applied to a practical recommendation task without any user behavioral data.

Works Cited

References

- Bird, S., Klein, E., and Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media.
- Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331–370.
- Celma, Ò., and Herrera, P. (2008). A new approach to evaluating novel recommendations. In *Proceedings of the 2008 ACM Conference on Recommender Systems (RecSys '08)* (pp. 179–186). ACM. <https://doi.org/10.1145/1454008.1454038>
- Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. (2020). A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*. PMLR.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (pp. 4171–4186). Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>
- Johnson, J., Douze, M., and Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>
- Koren, Y., Bell, R., and Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37. <https://doi.org/10.1109/MC.2009.263>
- McInnes, L., Healy, J., and Melville, J. (2018). UMAP: Uniform Manifold Approximation

and Projection for dimension reduction. *arXiv preprint arXiv:1802.03426*. <https://arxiv.org/abs/1802.03426>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., . . . Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP 2019)*. Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1410>

Schedl, M., Gómez, E., and Urbano, J. (2014). Music information retrieval: Recent developments and applications. *Foundations and Trends in Information Retrieval*, 8(2–3), 127–261. <https://doi.org/10.1561/15000000042>

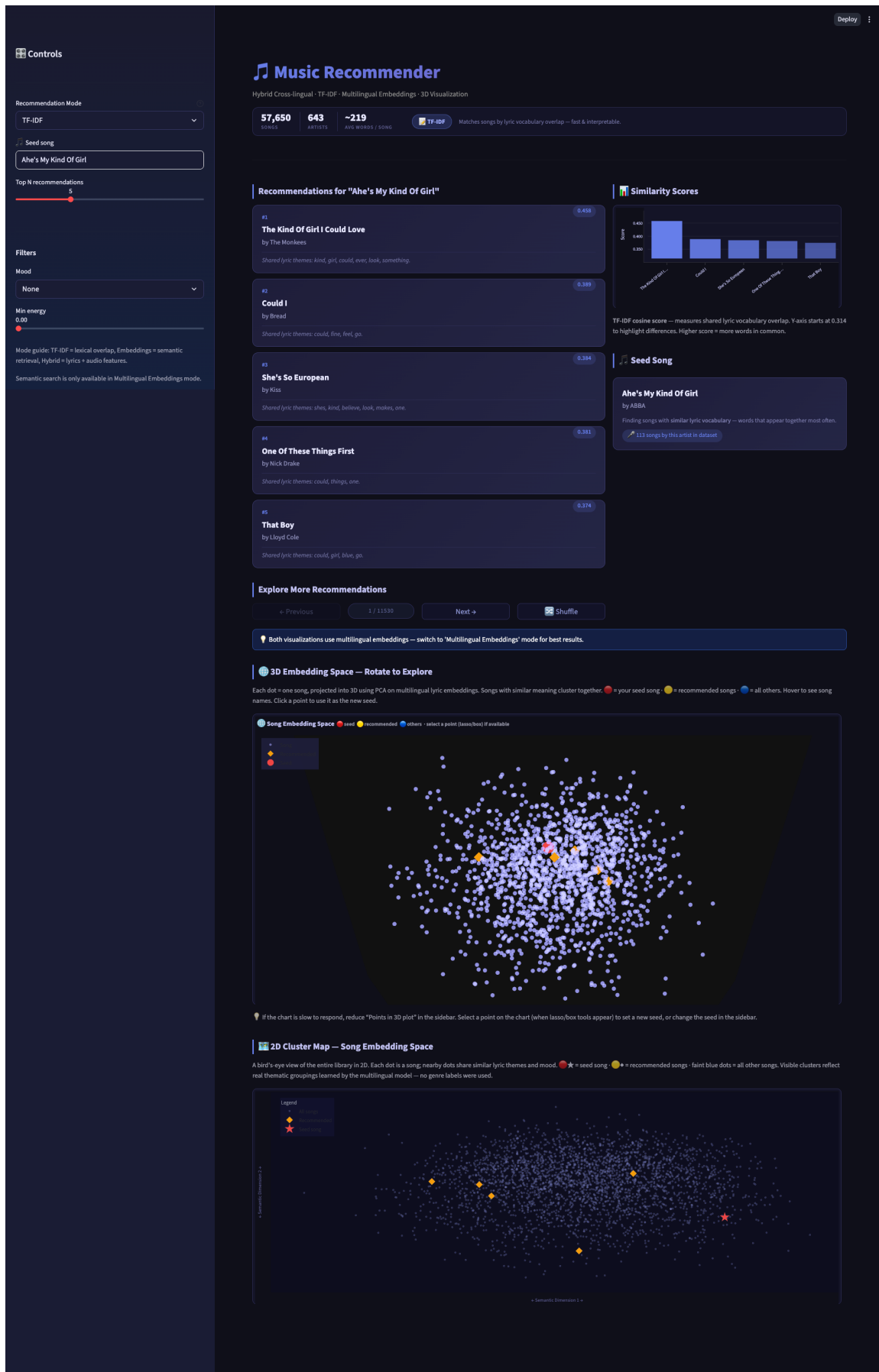


Figure 1: Main recommendation interface showing TF-IDF results with shared lyric theme explanations, similarity score bar chart, and seed song information panel.